COMP-1562 (Group 5)

OPERATING SYSTEMS Research and Technical tasks

By

Gizem Kumrili - 001313271-7

Michael Thomas - 001306638-2

James R Terris – 001394883-0

Shandip Chandra - 001379902-9

Abidur Rob - 001381097-9

Paaralathan Yasotharan - 001386934-5

(we have all contributed equally)

Technical Tasks

Task 1:

Task 1.1 - The hypothetical 16-bit processor

(Excel sheet attached to the submission area for task 1.1)

1.2 - Approximate Data Access Time

For this task we are going to be referencing a machine of the following specifications:

- 1K words Cache with an access time of 5ns
- 1M words memory with an access time of 60ns
- If the data is not in Cache, the data is copied from memory instead

Cache miss time

Cache miss time = Cache access time + Memory access time

For our machine, we have a cache access time of 5ns and a Memory access time of 60ns. When substituted into the equation, this gives us a Cache miss time of 65ns as shown below.

cache miss time = 5ns + 60ns

Cache miss time = 65ns

Average data access time (ADAT)

Average Data Access Time (ADAT) = Hit Ratio (H) * Cache hit + (1- H) * Cache Miss time

We are going to be calculating the average time taken for data to be accessed at 3 increasing hit ratios; 65%, 80% and 90%. Each equation will use the Cache miss time calculated prior of 65ns and the ADAT equation above

ADAT where Hit ratio = 65%:

ADAT = 0.65 * 5 + (1-0.65) * 65

ADAT = 0.65 * 5 + 0.35 * 65

ADAT = 26ns

ADAT where Hit ratio = 80%:

ADAT = 0.80 * 5 + (1-0.80) * 65

ADAT = 0.80 * 5 + (0.20) * 65

ADAT = 17ns

ADAT where Hit ratio = 90%:

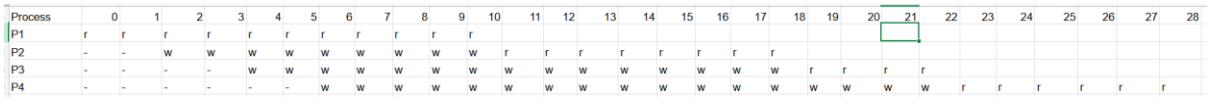
$$ADAT = 0.90 * 5 + (1-0.90) * 65$$

$$ADAT = 0.90 * 5 + (0.10) * 65$$

$$ADAT = 11ns$$

Task 2:

2.1 - First Come First Served



P1 Wait: 0

P2 Wait: 11 – 2 = 9

P3 Wait: 19 – 4 = 15

P4 Wait: 23 – 6 = 17

Average Waiting Time (tAWT): (0+9+15+17) / 4 = 10.25 ms

Average Turnaround Time (tATT}): (11+17+19+23) / 4 = 17.5 ms

2.2 - Shortest Job Next

P1 Wait: 0

P2 Wait: 21 – 2 = 19

P3 Wait: 11 – 4 = 7

P4 Wait:15 – 6 = 9

Average Waiting Time (t*AWT*): (0 + 19 + 7 + 9) / 4 = 8.75 ms

Average Turnaround Time (t*ATT*): (11 + 27 + 11 + 15) / 4 = 16 ms

Task 3:

3.1 - Scenario 1

First Fit: P1-M3,P2-M2,P3-M4,P4-M1,P5-M5

Best Fit: P1-M3,P2-M2,P3-M5,P4-M1,P5-M4

Next Fit: P1-M3,P2-M4,P3-M5,P4-M1,P5-M2

Worst Fit: P1-M3,P2-M4,P3-M5,P4-M2,P5-M1

3.2 - Scenario 2

First Fit: P1-M3,P2-M4,P3-M1,P4-M2,P5-M5

Best Fit: P1-M3,P2-M5,P3-M1,P4-M2,P5-M4

Next Fit: P1-M3,P2-M4,P3-M5,P4-M2,P5-M1

Worst Fit: P1-M3,P2-M4,P3-M5,P4-M2,P5-M1

Research-Based Tasks:

(References were used for further research and knowledge, our work is original. We conducted research, thought critically, shared personal reflection about the tasks collectively and drew conclusions.)

Scalability

Operating System	Scalability Features	Justification
Fuchsia	FIDL (Fuchsia Interface Definition Language)	This protocol allows quick and effective inter-process communication (IPC) with components systems that are separate. This allows the operating system to adapt and change from low energy gadgets to high-performance one without changing the communication methods between the components. This addresses the issue of maintaining software configurability across multiple hardware platforms. [1]
ROS	DDS (Data Distribution Service) Discovery	ROS 2 uses a decentralised discovery mechanism within its middleware rather than depending on a single master node. This removes the bottleneck found in prior versions and allows the system to expand to extensive multi-robot swarms. Consequently, it decreases network traffic and prevents singular failure points. [2]
DBOS	Distributed SQL (Based OS State)	DBOS keeps all operating system state, including process details and file metadata, within a high-performance distributed database. This allows the operating system to expand across thousands of cloud nodes while keeping strong consistency. It tackles the difficulty of handling a shared global state in an extensive computing setting. [3]

Security

Operating System	Security Property	Justification
Fuchsia	Sandboxing	Sandboxing follows the least privilege possible approach to defence. All programmes are executed within their own isolated environment without access

		to any resources to start only after creation does the OS assign resources to the program dependant on its requirements such as memory allocation or Kornel objects [4]. Utilising a Least privilege possible approach limits the spread of malware throughout the system by reducing the opportunities for lateral movement across the system to a smaller attack surface.
ROS	Confidentiality	The first version of ROS (ROS 1) didn't come with integrated security measures. Since then, more modern versions such as SROS have been secured with public key infrastructure (PKI) to encrypt network communications. All nodes part of SROS are issued a certificate from a certificate authority which the operating system can utilise to perform asymmetric encryption with the attached public key [5]. Encrypting data being used during network communication mitigates the risks of various attacks including main in the middle and packet sniffing by encoding the data being transferred to be unreadable without being decoded with the key.
DBOS	Access control	DBOS consists of authentications and permissions which a developer can implement within a programming environment to control and limit the access users have to the OS without the correct credentials such as a password [6]. The implementation of controls such as these can protect from unauthorised access which could lead to potential system intrusions and access to valuable or confidential data. With this data, an attacker could hold it for ransom, sell it to data brokers or delete it from systems managed by the OS, requiring rollbacks causing disruption and costing further resources.

Real-Time Processing

Operating System	Real-Time Processing Features	Justification
Fuchsia	Zircon Scheduling	Fuchsia is powered by Zircon, the core microkernel and is using "dynamic scheduling" as its primary scheduler. The difference between scheduling and dynamic scheduling is that normal scheduling will have fixed plans on tasks, making it inflexible. Dynamic scheduling in Zircon will

		continuously change/adapt its priorities based on the tasks and processes it has [9]. This may make repetitive tasks generally slower to complete, but under high loads or unpredictable tasks queued, it is an improvement for real-time processing.
ROS	Data Distribution Service (DDS)	An improvement to ROS1 to ROS2 is its movement to a DDS, designed to provide higher efficiency, reliability and low latency. One specific way is that it provides shared memory to optimize message traffic whereas ROS1 never had that standard. This boosts intraprocess communications (exchange of data and action coordination) and improves real-time processing [10]
DBOS	Durable Queues	One of the main features of durable queues is that you can enqueue (add more to a queue) “workflows for distributed execution and flow control.” [11] This means tasks are worked on at the same time, decreasing overall processing time. DBOS queues also allow the system to keep a list of workflow handles, giving the system or user the ability to cancel or modify tasks which can be used to decrease process times of other tasks. [12]

Application

Operating System	Application Features	Justification
Fuchsia	Component-Based Sandboxing	Fuchsia idea is that "everything is a component" works for consumer IoT. Because software is distributed in hermetic (self-contained) packages, the OS may update certain drivers or household automation modules without needing a full system reboot. This takes advantage of the Zircon kernel's ability to provide 100% uptime for important security monitoring while updating UI components in the background. [7]
ROS	DDS Middleware & QoS Profiles	Autonomous robots need strong integration of sensors (LiDAR, Radar, and Cameras). ROS 2's usage of Data Distribution Service (DDS) allows the AI components to communicate with hardware components which ensures the quality of service. This means a developer may prioritise collision avoidance data over battery status data, ensuring that the AI agent receives important information with expected delay. [8]
DBOS	Exactly-Once Transactional State	In machine learning training, data input is often the biggest obstacle. DBOS treats all OS activities as database transactions. This "provenance" allows a pipeline to give "exactly-once" operation. If a node dies during a huge data-cleaning activity, DBOS can accurately go back or continue

		from the last recorded state, removing corrupted or duplicate data, which often destroys AI model training. [3]

Bibliography

1. [1] Fuchsia. (2026). *FIDL Overview*. [online] Available at: https://fuchsia.dev/fuchsia-src/concepts/fidl/overview [Accessed 13 Feb. 2026].
2. [2] Macenski, S., Foote, T., Gerkey, B., Lalancette, C. and Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66). doi:https://doi.org/10.1126/scirobotics.abm6074.
3. [3]Skiadopoulos, A., Li, Q., Kraft, P., Kaffes, K., Hong, D., Mathew, S., Bestor, D., Cafarella, M., Gadepally, V., Graefe, G., Kepner, J., Kozyrakis, C., Kraska, T., Stonebraker, M., Suresh, L. and Zaharia, M. (2021). DBOS. *Proceedings of the VLDB Endowment*, 15(1), pp.21–30. doi:https://doi.org/10.14778/3485450.3485454.
4. [4] Pagano, F., Verderame, L. and Merlo, A. (2021). Understanding Fuchsia Security. *arXiv.org*. [online]: https://doi.org/10.22667/JOWUA.2021.09.30.047
5. [5] T. Mokhamed, F. M. Dakalbab, S. Abbas and M. A. Talib, "Security in Robot Operating Systems (ROS): analytical review study," The 3rd International Conference on Distributed Sensing and Intelligent Systems (ICDSIS 2022), Hybrid Conference, Sharjah, United Arab Emirates, 2022, pp. 79-94, doi: 10.1049/icp.2022.2422. Available at: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10017240
6. [6] Li, Q., Kraft, P., Kozyrakis, C., Zaharia, M. and Stonebraker, M. (2025). DBOS: three years later. *The VLDB Journal*, 34(3). Available at: https://doi.org/10.1007/s00778-024-00899-0
7. [7] Fuchsia. (2025). *Introduction to Fuchsia components*. [online] Available at: https://fuchsia.dev/fuchsia-src/concepts/components/v2/introduction.
8. [8] Ros.org. (2025). *Different ROS 2 middleware vendors — ROS 2 Documentation: Rolling documentation*. [online] Available at: https://docs.ros.org/en/rolling/Concepts/Intermediate/About-Different-Middleware-Vendors.html.
9. [9] Suann, J. (n.d.). *Zircon on seL4*. [online] Available at: https://trustworthy.systems/publications/theses_public/18/Suann%3Abe.pdf.
10. [10] Lienen, C., Middeke, S.H. and Platzner, M. (2023). FPGADDS: An Intra-FPGA Data Distribution Service for ROS 2 Robotics Applications. *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, [online] pp.6261–6266. doi:https://doi.org/10.1109/iros55552.2023.10341921.
11. [11]Dbos.dev. (2026). *DBOS Architecture | DBOS Docs*. [online] Available at: https://docs.dbos.dev/architecture.
12. [12] Vasquez-Grinnell, S. and Poliakov, A. (2025). *S3Mirror: Making Genomic Data Transfers Fast, Reliable, and Observable with DBOS*. [online] arXiv.org. Available at: https://arxiv.org/abs/2506.10886 [Accessed 19 Feb. 2026].